

队列

队列接口与实现

操作与接口

❖ 队列 (queue) 也是受限的序列

❖ 只能在队尾插入 (查询) :

- enqueue() / rear()

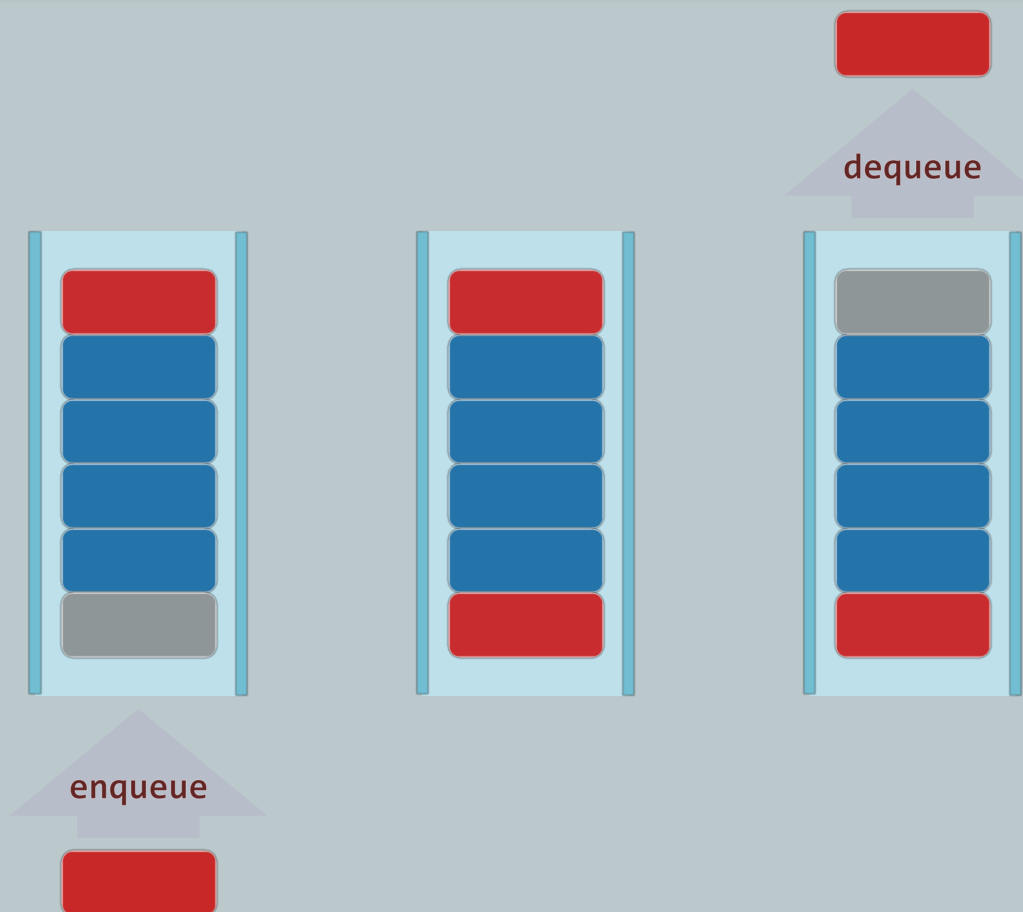
❖ 只能在队头删除 (查询) :

- dequeue() / front()

❖ 先进先出 (FIFO)

后进后出 (LILO)

❖ 扩展接口: getMax()...



实例

操作	输出	队列（右侧为队头）		
Queue()				
empty()	true			
enqueue(5)		5		
enqueue(3)		3	5	
dequeue()	5	3		
enqueue(7)		7	3	
enqueue(3)		3	7	3
front()	3	3	7	3
empty()	false	3	7	3

操作	输出	队列（右侧为队头）					
enqueue(11)		11	3	7	3		
size()	4	11	3	7	3		
enqueue(6)		6	11	3	7	3	
empty()	false	6	11	3	7	3	
enqueue(7)		7	6	11	3	7	3
dequeue()	3	7	6	11	3	7	
dequeue()	7	7	6	11	3		
front()	3	7	6	11	3		
size()	4	7	6	11	3		

实现

❖ 队列既然属于序列的特例，故亦可直接基于向量或列表派生

❖ `template <typename T> class Queue: public List<T> {`

`public: //原有接口一概沿用`

`void enqueue( T const & e ) { insertAsLast( e ); } //入队`

`T dequeue() { return remove( first() ); } //出队`

`T & front() { return first()->data; } //队首`

`}; //以列表首/末端为队列头/尾——颠倒过来呢?`

❖ 确认：如此实现的队列接口，均只需 $O(1)$ 时间

❖ 课后：基于向量派生实现队列模板类，并就其效率做一评估

队列

队列应用

资源循环分配

❖ 一组客户 (client) 共享同一资源时, 如何兼顾**公平与效率**?

比如, 多个应用程序共享CPU, 实验室成员共享打印机, ...

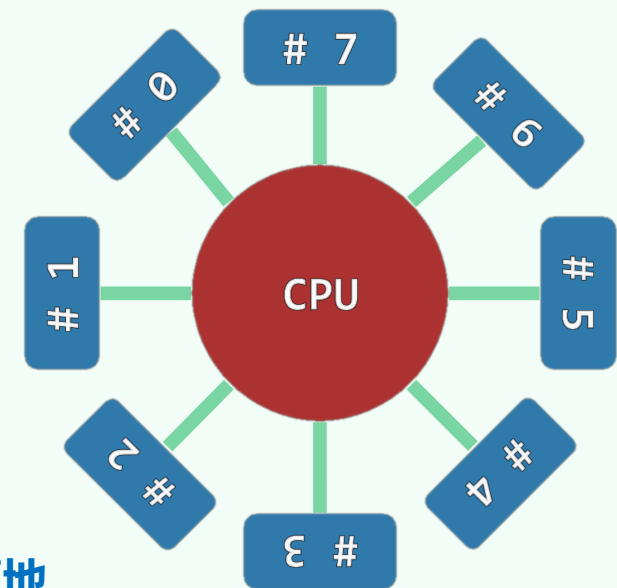
❖ RoundRobin //循环分配器

```
Queue Q( clients ); //共享资源的所有客户组成队列
```

```
while ( ! ServiceClosed() ) //在服务关闭之前, 反复地
```

```
    e = Q.dequeue(); //令队首的客户出队, 并
```

```
    serve( e ); Q.enqueue( e ); //接受服务, 然后重新入队
```



银行服务模拟：模型

❖ 提供n个服务窗口

- 任一时刻，每个窗口至多接待一位顾客
其他顾客排队等候
- 顾客到达后，自动地
选择和加入最短队列（的末尾）

❖ 参数： nWin //窗口（队列）数目

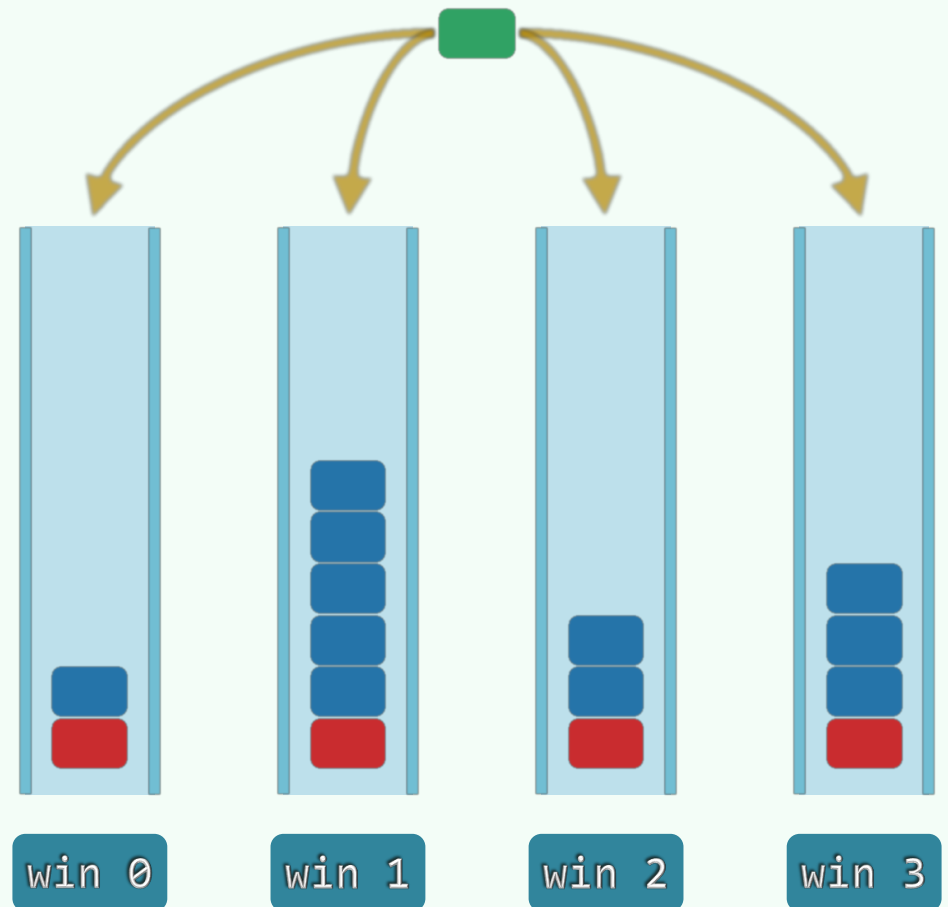
servTime //营业时长

❖ struct Customer { //顾客类

int window; //所属窗口（队列）

unsigned int time; //服务时长

};

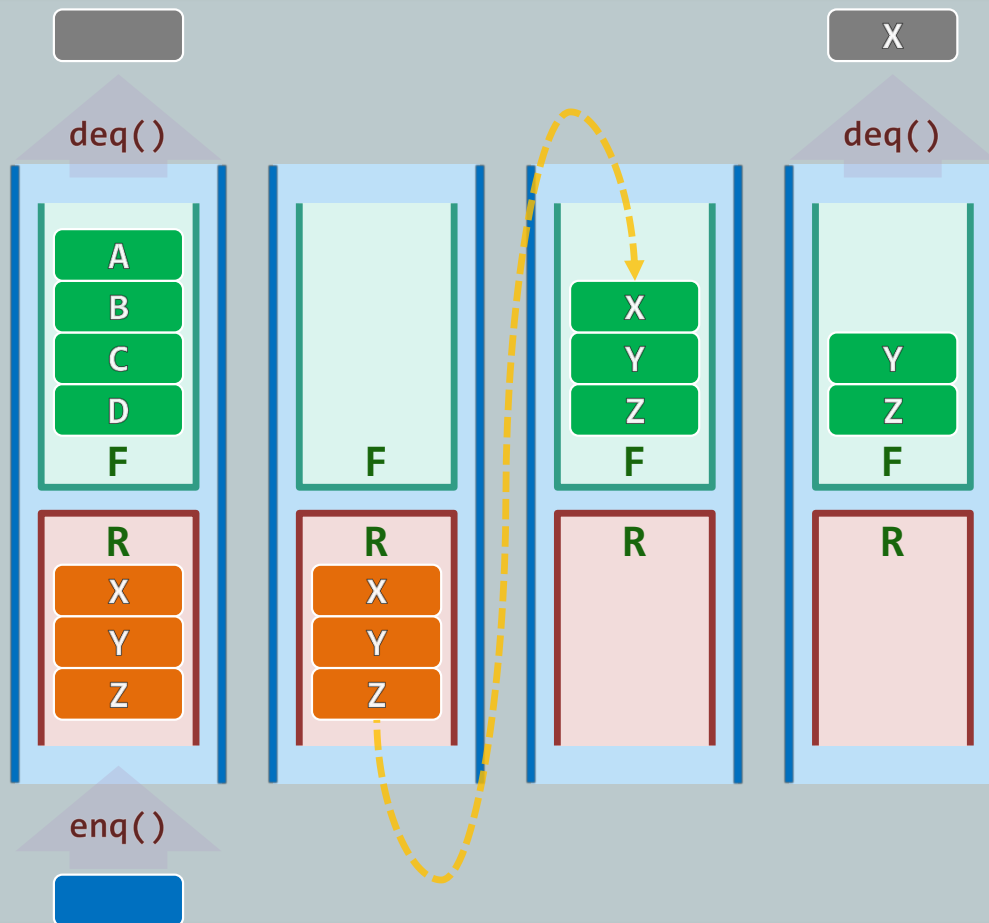


银行服务模拟：实现

```
❖ void simulate( int nWin, int servTime ) {  
    Queue<Customer> * windows = new Queue<Customer>[ nWin ];  
    for ( int now = 0; now < servTime; now++ ) { //在下班之前，每隔单位时间  
        Customer c ; c.time = 1 + rand() % 50; //一位新顾客到达，其服务时长随机指定  
        c.window = bestWindow( windows, nWin ); //找出最佳（最短）服务窗口  
        windows[ c.window ].enqueue( c ); //新顾客加入对应的队列  
        for ( int i = 0; i < nWin; i++ ) //分别检查  
            if ( ! windows[ i ].empty() ) //各非空队列  
                if ( -- windows[ i ].front().time <= 0 ) //队首顾客接受服务  
                    windows[ i ].dequeue(); //服务完毕则出列，由后继顾客接替  
    } //for  
    delete [] windows; //释放所有队列  
}
```

队列
双栈当队

Queue = Stack x 2

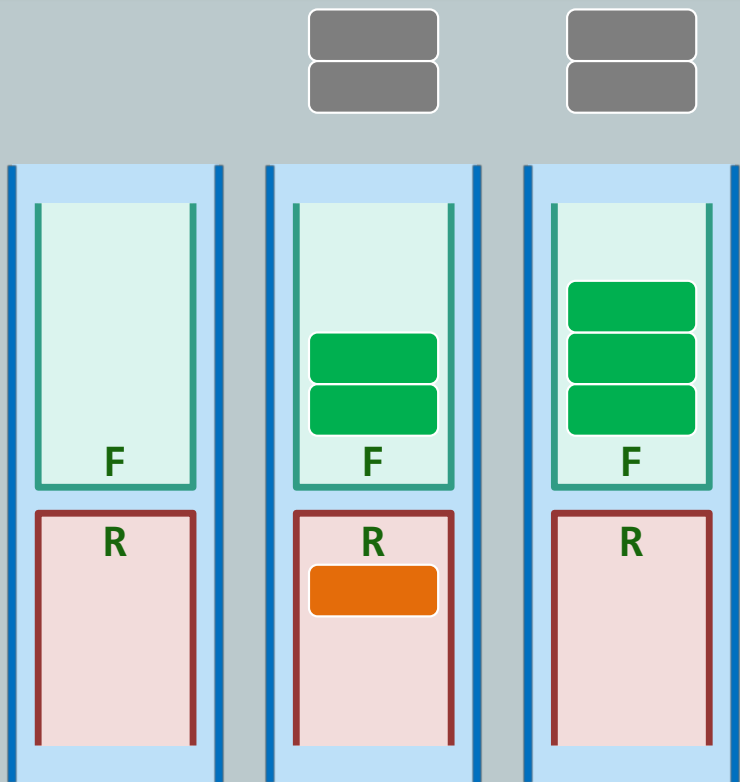


```
❖ def Q.enqueue(e)
    R.push(e);

❖ def Q.dequeue() # 0 < Q.size()
    if ( F.empty() )
        while ( !R.empty() )
            F.push( R.pop() );
    return F.pop();

❖ Best/worst case:  $O(1)/O(n)$ 
    Average? Amortization!
```

Amortization By Aggregate



❖ Consider the moment when

d dequeue()'s and e enqueue()'s

have been done $// d \leq e$

❖ The time cost

for **ALL** the operations is

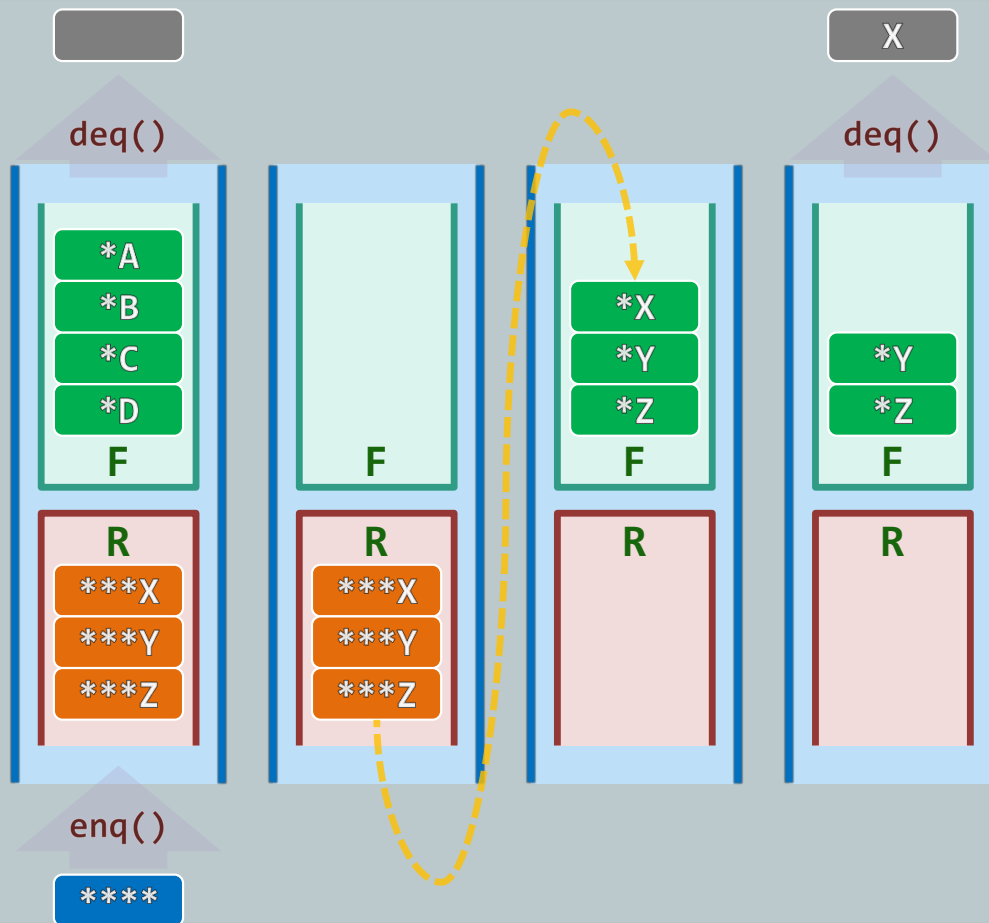
$$\leq 4 \cdot d + 3 \cdot (e - d) = 3e + d$$

❖ The amortized cost

for each OPERATION is

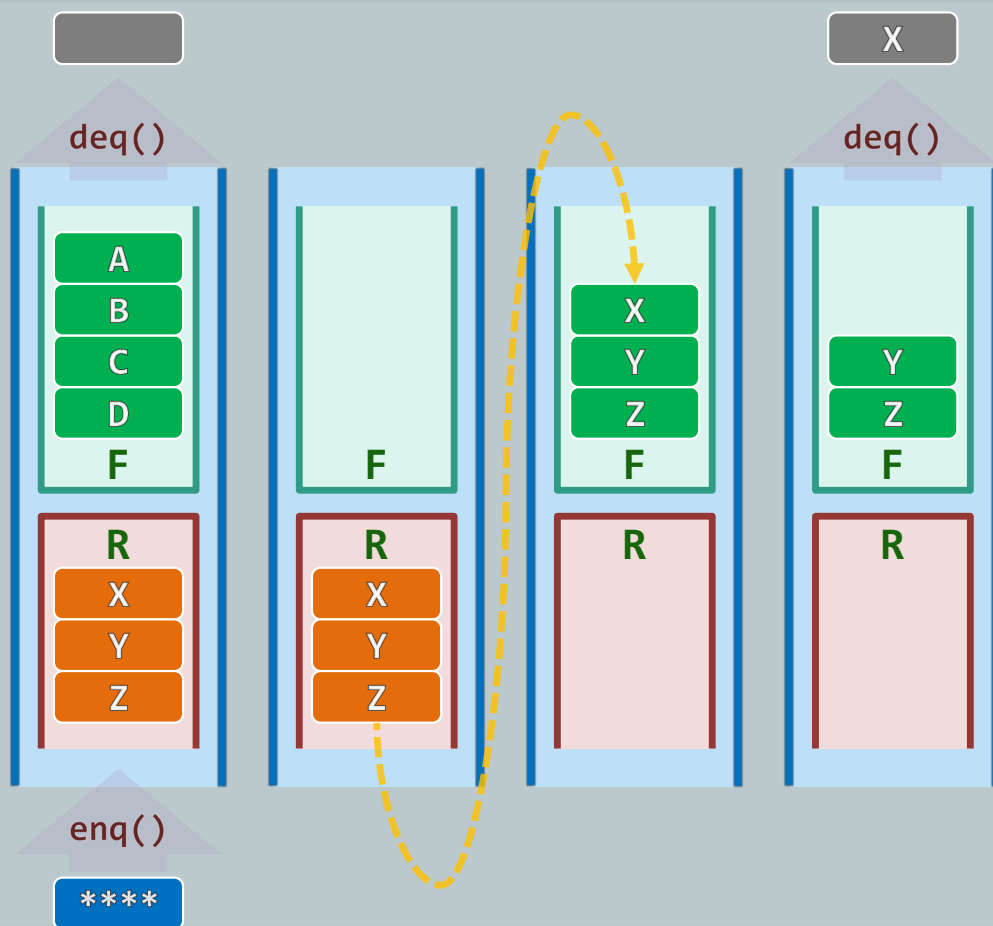
$$\frac{3e + d}{e + d} < 3$$

Amortization By Accounting



- ❖ Assign each new element with **4 coins** `//deposit`
 - **1** for its `enqueue()`
 - **2** for transfer, and
 - the last **1** for `dequeue()`
- ❖ Hence every operation is **pre-paid** and ...
- ❖ The structure will never run out of credit
- ❖ **Amortized cost** of any operation sequence involving **`n`** ITEMS is
$$4n = O(n)$$

Amortization By Potential



❖ Consider the k^{th} operation

❖ Define $\Phi_k = |R_k| - |F_k|$

❖ Then $A_k = T_k + \Phi_k - \Phi_{k-1} \equiv 2$

❖ Hence

$$2n \equiv \sum_{k=1}^n A_k = \sum_{k=1}^n T_k + \Phi_n - \Phi_0$$

$$2n = T(n) + \Phi_n - \Phi_0 > T(n) - n$$

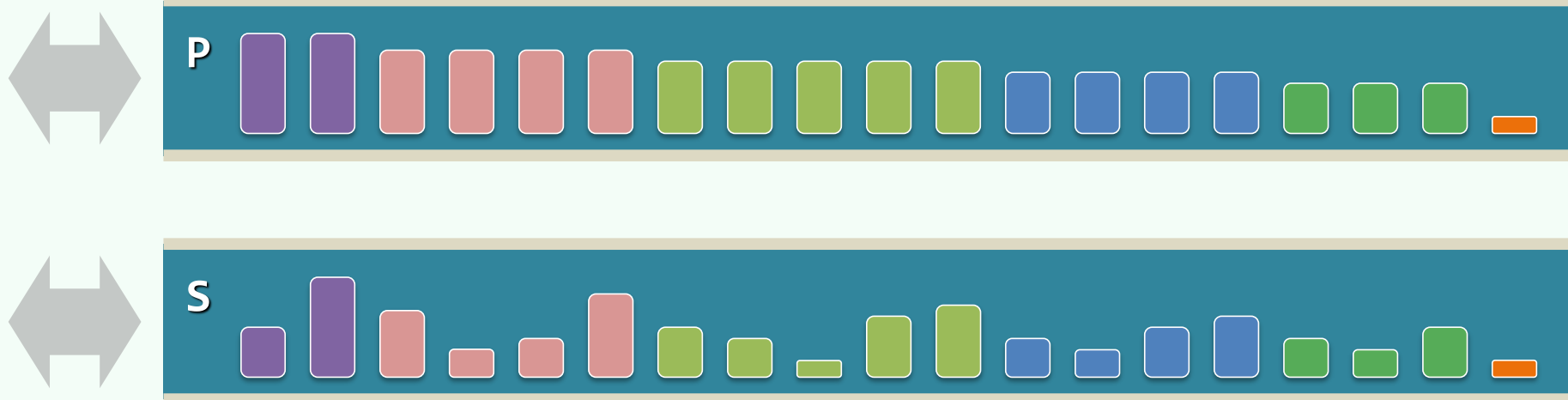
$$T(n) < 3n = \mathcal{O}(n)$$

队列

Steap+Queap

Steap = Stack + Heap = push + pop + getMax

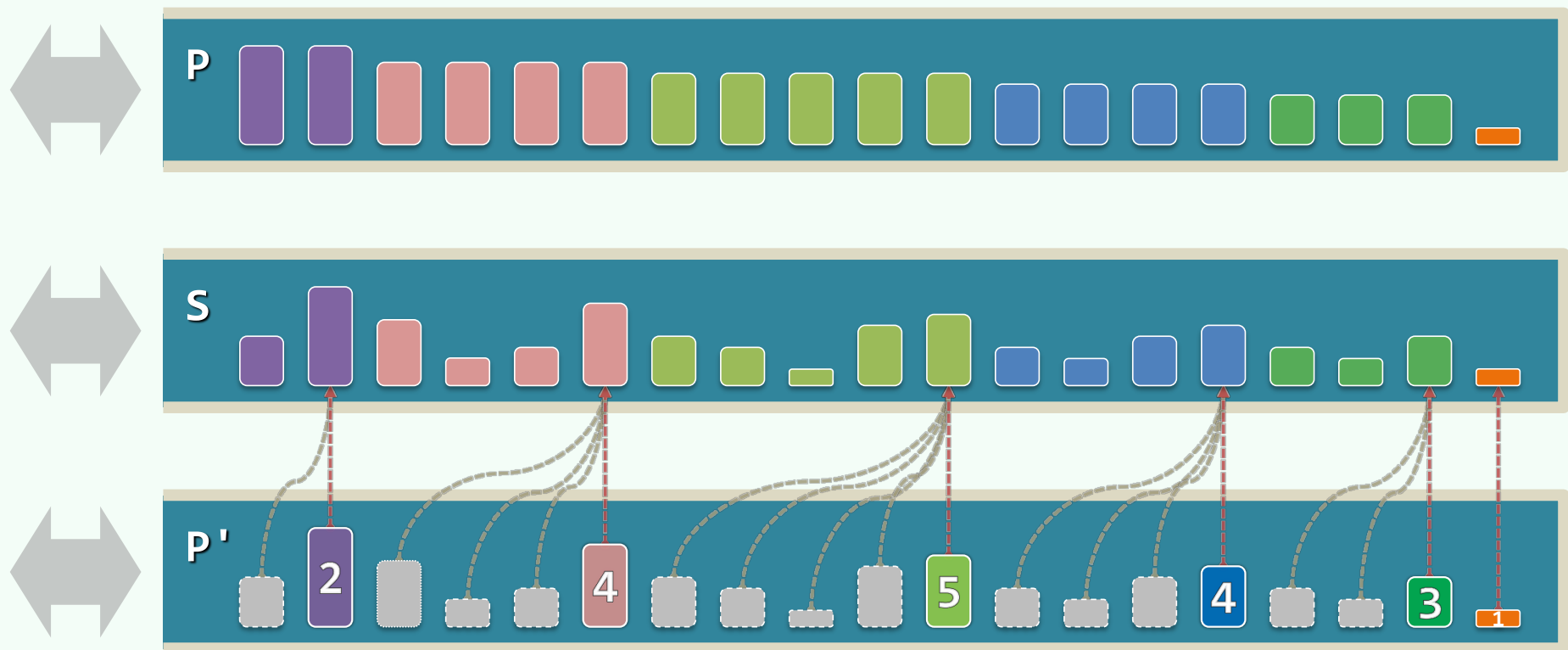
❖ P中每个元素，都是S中对应**后缀**里的**最大者**：Steap::getMax() { return P.top(); }



❖ Steap::pop() { P.pop(); return S.pop(); } *//O(1)*

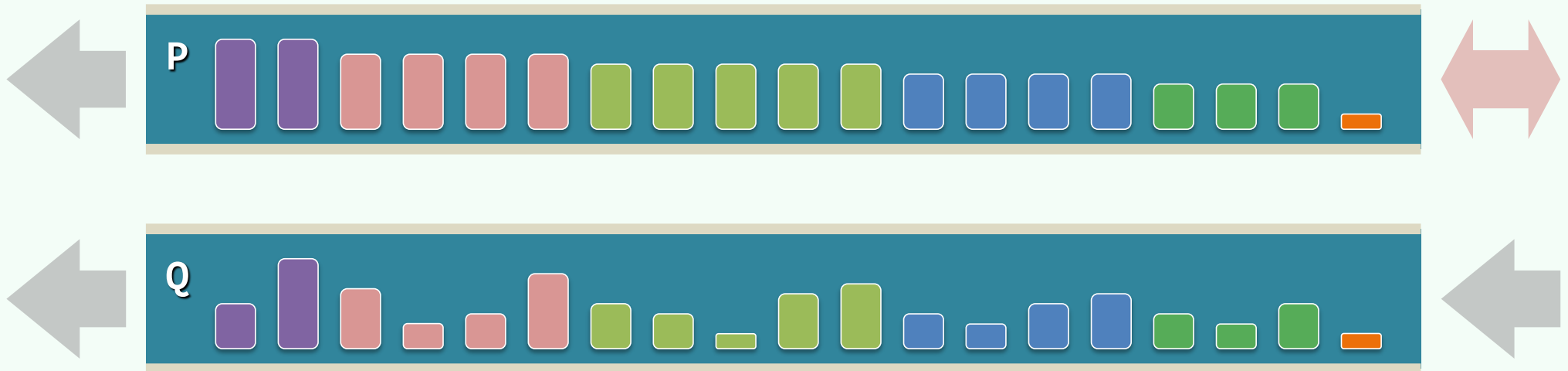
❖ Steap::push(e) { P.push(max(e, P.top())); S.push(e); } *//O(1)*

Steap = Stack + Heap = push + pop + getMax



Queap = Queue + Heap = enqueue + dequeue + getMax

❖ `Queap::dequeue()` { `P.dequeue()`; `return Q.dequeue()`; } *// $O(1)$*



❖ `Queap::enqueue(e)` {

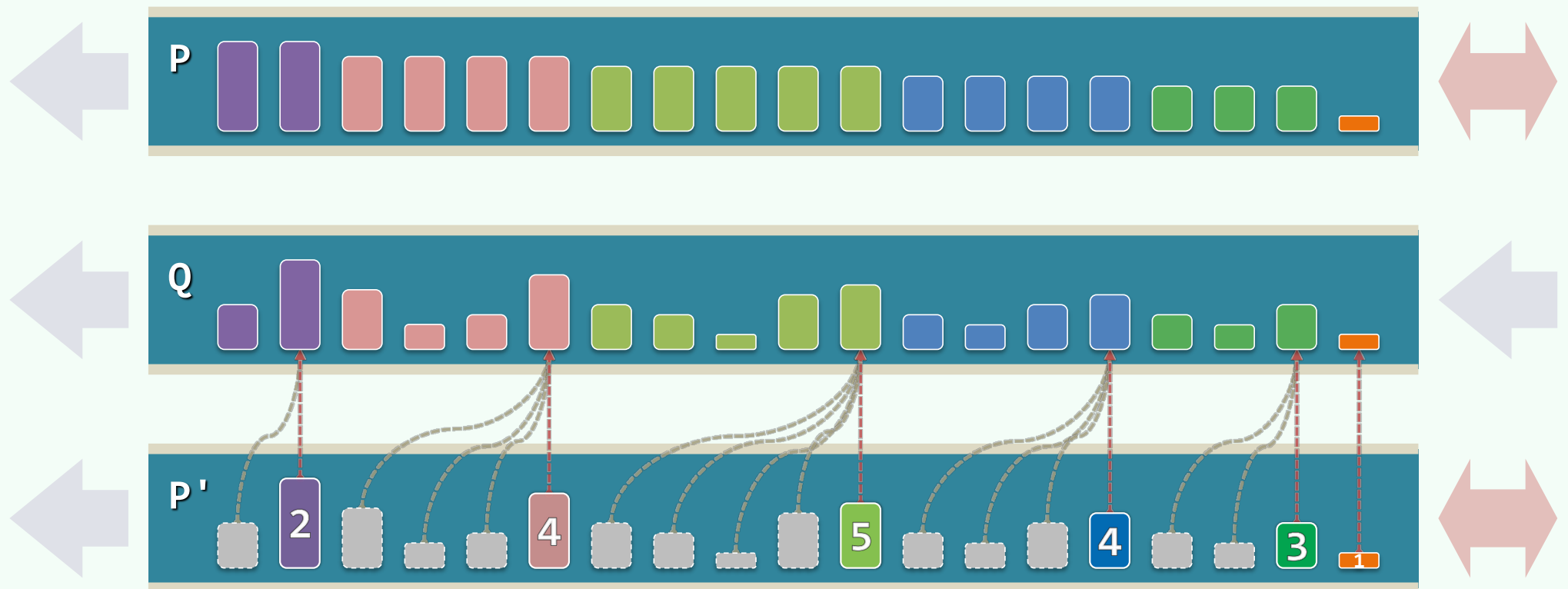
`Q.enqueue(e)`; `P.enqueue(e)`;

for (`x = P.rear()`; `x` && (`x->key` $\leq$ `e`); `x = x->pred`) *//最坏情况 $O(n)$*

`x->key = e`;

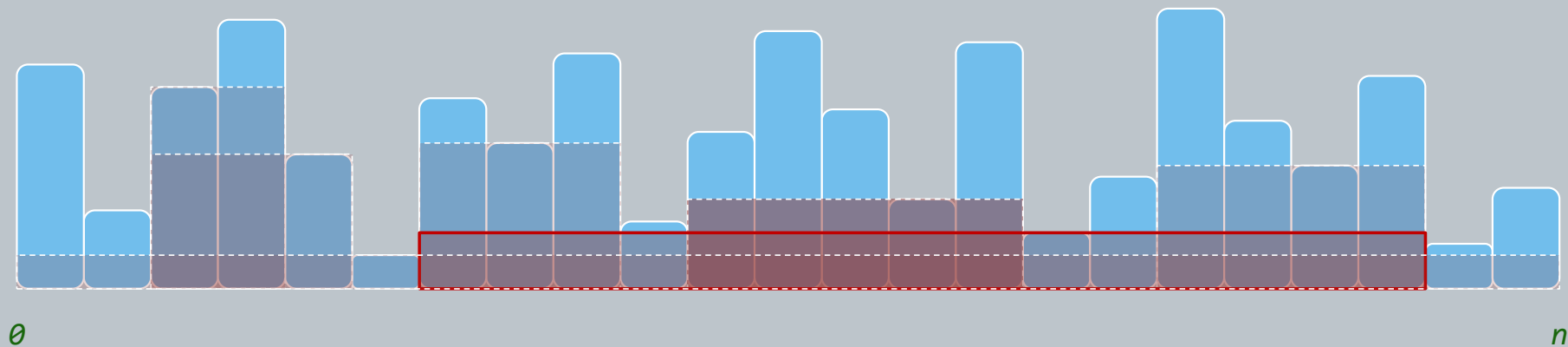
}

Queap = Queue + Heap = enqueue + dequeue + getMax



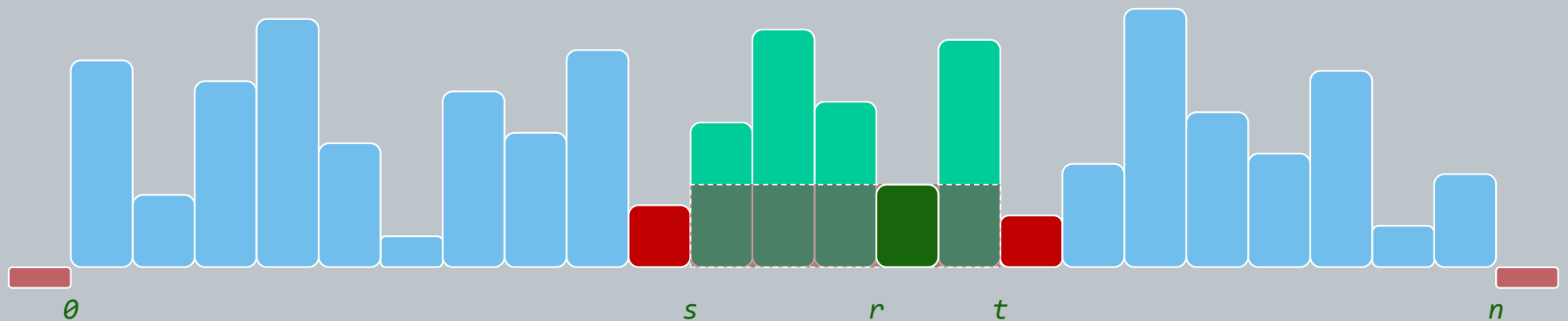
队列
直方图内最大矩阵

Maximum Rectangle



- ❖ Let $H[0,n)$ be a histogram of non-negative integers
- ❖ How to find the largest orthogonal rectangle in $H[]$?
- ❖ To eliminate possible ambiguity
we can, for example, choose the **LEFTMOST** one

Maximal Rectangles



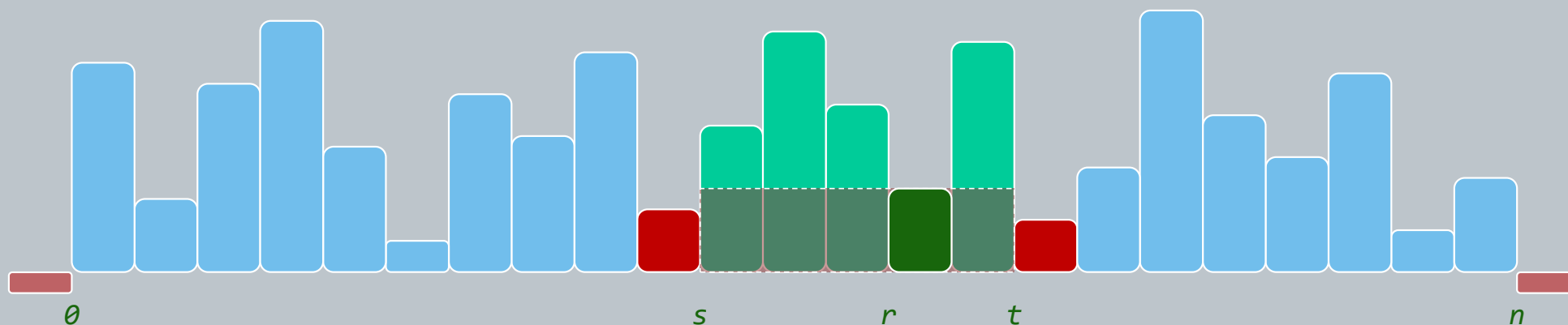
❖ Maximal rectangle supported by $H[r]$: $maxRect(r) = H[r] \cdot (t(r) - s(r))$

$$s(r) = \max\{ k \mid 0 \leq k < r \text{ and } H[k-1] < H[r] \}$$

where

$$t(r) = \min\{ k \mid r < k \leq n \text{ and } H[r] > H[k] \}$$

Brute-force



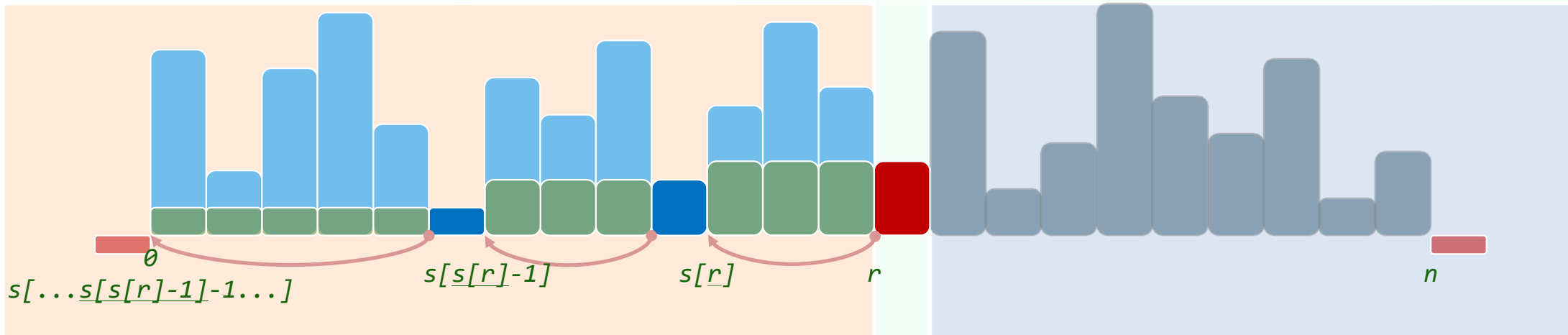
❖ Determining $s(r)$ and $t(r)$ for all r 's requires $\mathcal{O}(n^2)$ time

$$s(r) = \max\{ k \mid 0 \leq k < r \text{ and } H[k-1] < H[r] \}$$

$$t(r) = \min\{ k \mid r < k \leq n \text{ and } H[r] > H[k] \}$$

❖ Actually, we can do this more efficiently ...

Using Stack: Algorithm



```
Rank* s = new Rank[n]; Stack<Rank> S;
```

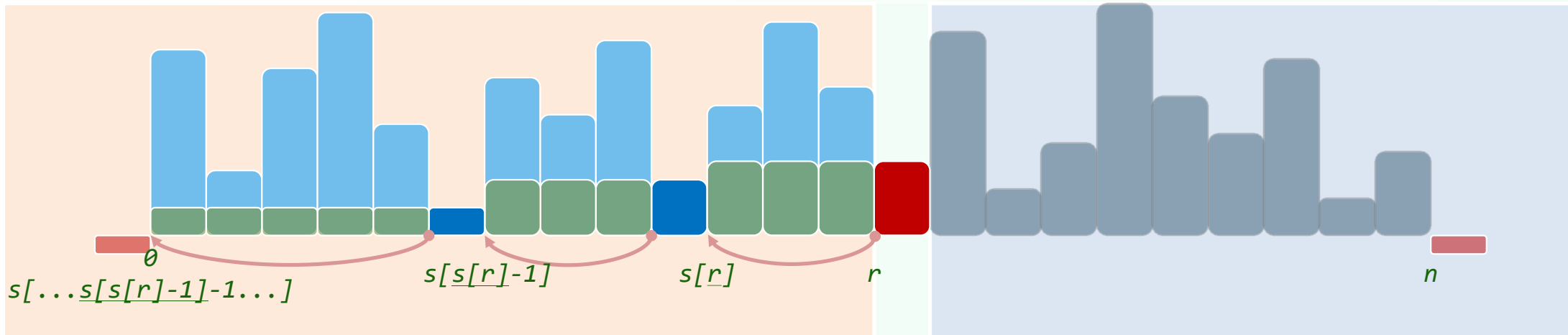
```
for ( Rank r = 0; r < n; r++ ) //try using SENTINEL for simplicity by yourself
```

```
    while ( !S.empty() && ( H[S.top()] >= H[r] ) ) S.pop(); //until H[top] < H[r]
```

```
    s[r] = S.empty() ? 0 : 1 + S.top();    S.push(r); //S is always ASCENDING
```

```
while( !S.empty() ) S.pop();
```

Using Stack: Loop Invariant & Correctness



❖ All bars are scanned and pushed into S **in turn**

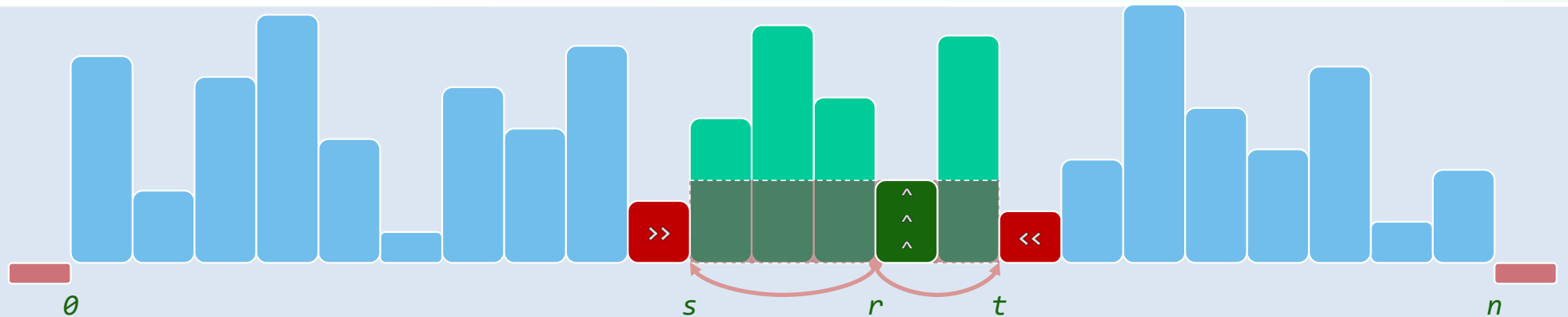
❖ After each iteration of the outer loop, S stores a **chain** in terms of $s[]$

$$S[S.size() - 1] = S.top() = r \quad \text{and} \quad \forall 0 \leq k < S.size(), \quad S[k - 1] + 1 = s[S[k]]$$

❖ Every bar is popped when it will be of **no use** thereafter, i.e.

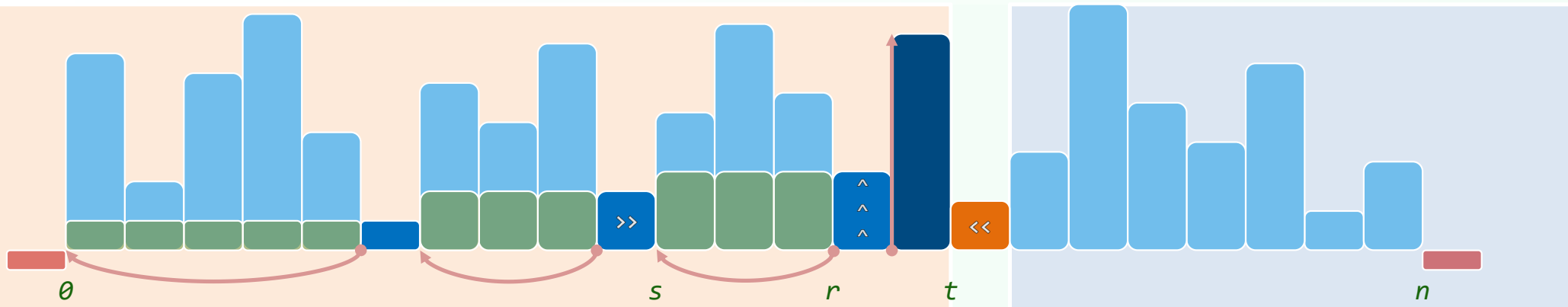
$$\forall 0 \leq k < r \text{ but } k \notin S, \quad \nexists r \leq t \text{ s.t. } k + 1 = s(t)$$

Using Stack: Complexity



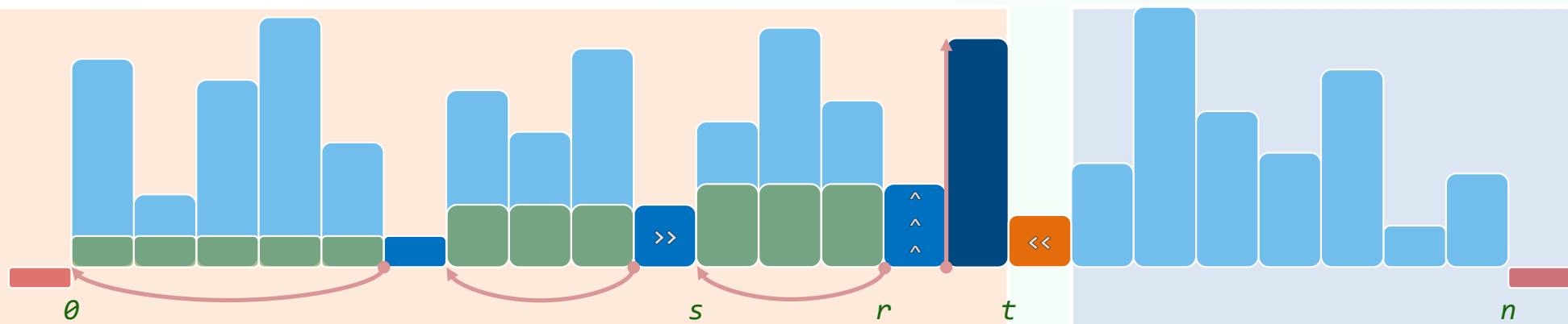
- ❖ And $t(r)$'s can be determined by another scan in the **REVERSED** direction
 - ❖ Hence all maximal rectangles can be computed in $\mathcal{O}(n)$ time and using $\mathcal{O}(n)$ space
 - ❖ However, what if the histogram is given in an **IN-PLACE** and **ON-LINE** manner?
- Note that, the $t(r)$'s **CAN'T** be determined until the **ENTIRE** input is ready
- ❖ Is it possible to compute **BOTH** $s(r)$'s and $t(r)$'s by a **SINGLE** scan? *//on-fly*

One-Pass Scan: Algorithm



```
Stack<Rank> SR;  __int64 maxRect = 0; //SR.2ndTop() == s(r)-1 & SR.top() == r
for ( Rank t = 0; t <= n; t++ ) //amortized- $\mathcal{O}(n)$ 
    while ( !SR.empty() && ( t == n || H[SR.top()] > H[t] ) )
        Rank r = SR.pop(), s = SR.empty() ? 0 : SR.top() + 1;
        maxRect = max( maxRect, H[r] * ( t - s ) );
    if ( t < n ) SR.push( t );
return maxRect;
```

One-Pass Scan: Loop Invariant & Correctness



❖ Again, **at** each iteration of the **outer** loop, we always have

$$\forall 0 \leq k < SR.size(), \quad SR[k-1] + 1 = s[SR[k]]$$

❖ For each bar r **popped** in the **inner** loop, we have

$$t(r) = t \quad \text{and} \quad s[r] = SR.top() + 1$$